Частное учреждение профессионального образования «Высшая школа предпринимательства»

КУРСОВОЙ ПРОЕКТ

Разработка реляционной базы данных

для конструктора ПВЗ

Студент: **Воробьёв Роман Алексеевич** • 3-й курс

Специальность: **09.02.07 «Информационные системы и программирование»**

Преподаватель: **к.ф.н. Ткачев П.С.**

2026

ЦЕЛЬ И ЗАДАЧИ ПРОЕКТА

ЦЕЛЬ

Разработать реляционную базу данных для мультитенантной платформы конструктора пунктов выдачи заказов (ПВЗ), отражающую ключевые бизнес-процессы платформы электронной торговли «Django Marketplace».

ЗАДАЧИ

01 Анализ бизнес-процессов

Выявить и описать бизнес-процессы мультитенантной платформы

02 Требования к БД

Сформулировать функциональные и нефункциональные требования

03 Выбор СУБД

Обосновать выбор оптимальной СУБД для платформы

04 ER-диаграмма

Построить концептуальную и логическую модель данных

05 Физическая структура

Разработать физическую структуру: таблицы, связи, ограничения

06 Бизнес-логика

Реализовать триггеры, процедуры, функции, представления, роли

07 Тестирование

Проверить реализованные механизмы на тестовых данных

ПРЕДМЕТНАЯ ОБЛАСТЬ: КОНСТРУКТОР ПВЗ

Что такое конструктор ПВЗ?

Программная платформа класса SaaS с мультитенантной архитектурой. Каждый арендатор получает независимый интернет-магазин с собственным доменом, каталогом товаров и сетью ПВЗ.

Участники системы:

Суперадминистратор
Управляет платформой, настраивает тенантов

Арендатор (ПВЗ-магазин)
Независимый магазин с доменом и каталогом

Менеджер ПВЗ
Обрабатывает заказы, управляет магазином

Клиент
Конечный покупатель, совершает покупки

Актуальность

20–25%
ежегодный рост e-commerce в России *АКИТ, 2025*

SaaS + Multi-tenant
архитектура: один код - тысячи магазинов

14 таблиц
спроектированных в нормальной форме 3NF

MySQL 8.0
выбранная СУБД - открытый исходный код

КЛЮЧЕВЫЕ БИЗНЕС-ПРОЦЕССЫ ПЛАТФОРМЫ

tenant

Управление арендаторами

Регистрация ПВЗ-магазинов, домен, адрес, режим работы, статус активности

auth_user

Управление пользователями

Роли: customer / manager / admin. Профили, разграничение прав доступа

product

Каталог товаров

Мультитенантный каталог: бренды, категории, товары, остатки, JSON-атрибуты

cart

Корзина покупателя

Добавление товаров, изменение количества, привязка к сессии / пользователю

order

Жизненный цикл заказа

pending → confirmed → shipped → delivered → completed / cancelled

payment

Обработка платежей

Разные методы оплаты, статус транзакции, время оплаты, привязка к заказу

site_config

Конструктор сайта

Site Builder: создание страниц drag-and-drop, хранение конфигурации в JSON

notification

Уведомления и мониторинг

Системные уведомления, отслеживание посылок (просрочка >7 дней на ПВЗ)

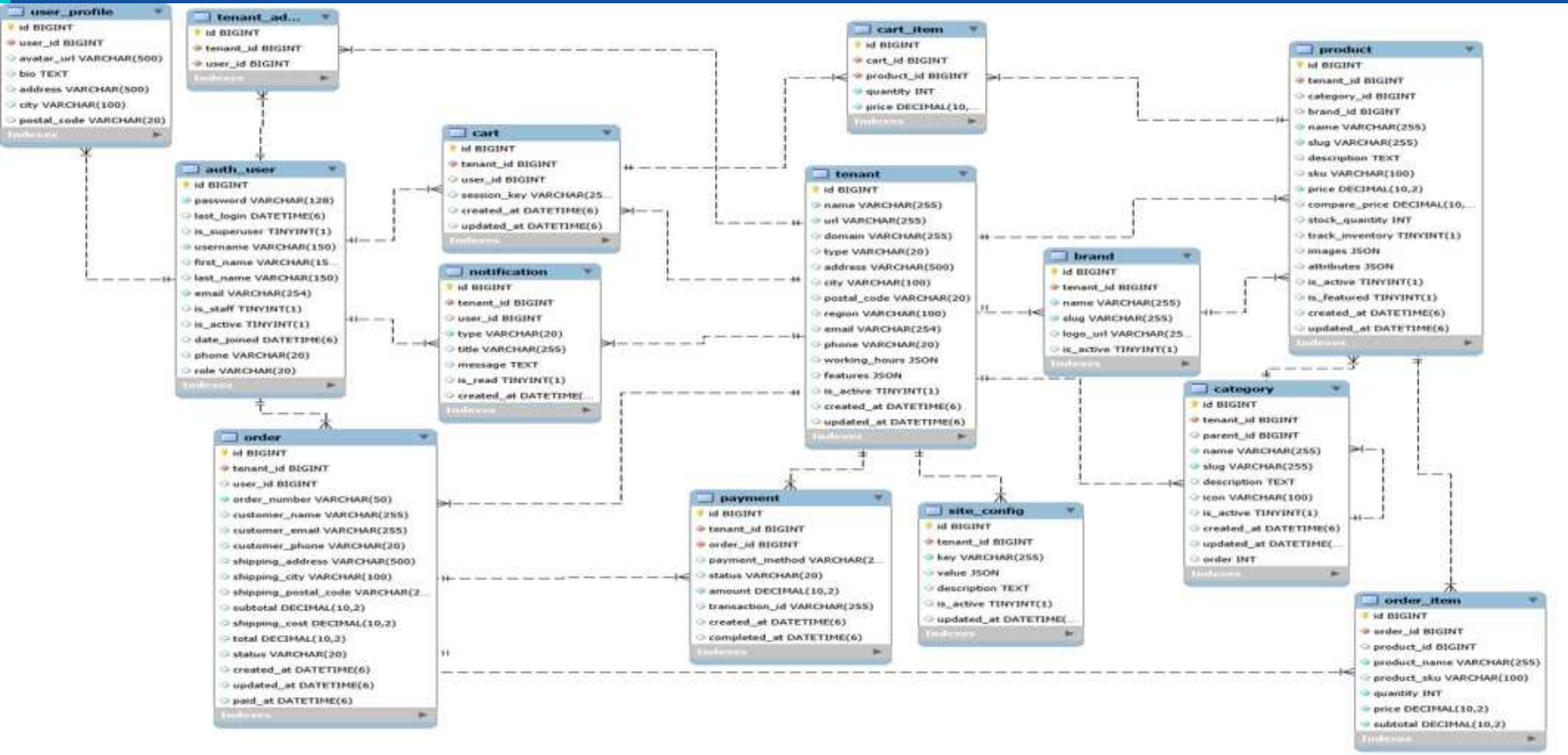
ВЫБОР СУБД: MySQL vs PostgreSQL

Критерий оценки	MySQL 8.0	PostgreSQL 15
OLTP-производительность	Высокая (InnoDB)	Высокая
JSON-поля	Поддерживается	Расширенная (jsonb)
Полнотекстовый поиск	Встроенный	Встроенный
Совместимость Django ORM	Нативная	Нативная
Репликация / HA	Group Replication	Streaming Replication
Триггеры / Процедуры	Полная поддержка	Полная поддержка
Простота настройки	Высокая	Средняя
Опыт работы с СУБД	Пройден курс **(МДК.11.01) и (ОП 08)**	Ограниченный

✓ Выбрана СУБД MySQL 8.0

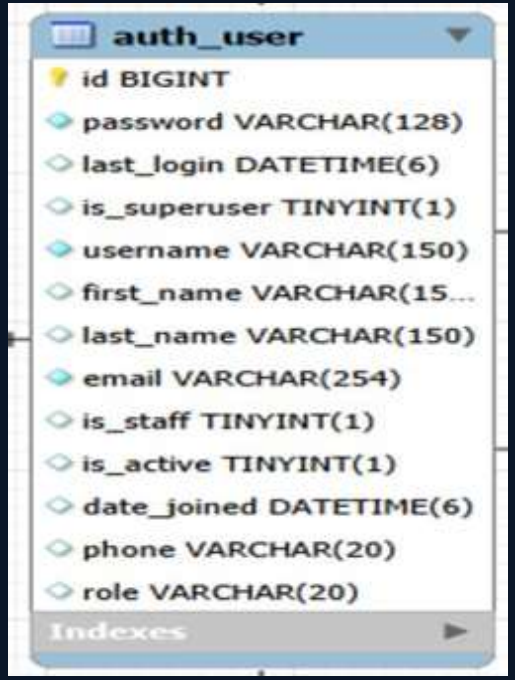
Оптимальное сочетание производительности, совместимости с Django ORM и опыта команды. Все функциональные требования (триггеры, процедуры, JSON, роли) реализуются штатными средствами MySQL.

ER-ДИАГРАММА БАЗЫ ДАННЫХ (MySQL Workbench)



СТРУКТУРА ТАБЛИЦ: ПОЛЬЗОВАТЕЛИ И АРЕНДАТОРЫ

auth_user — пользователи системы



user_profile — профили

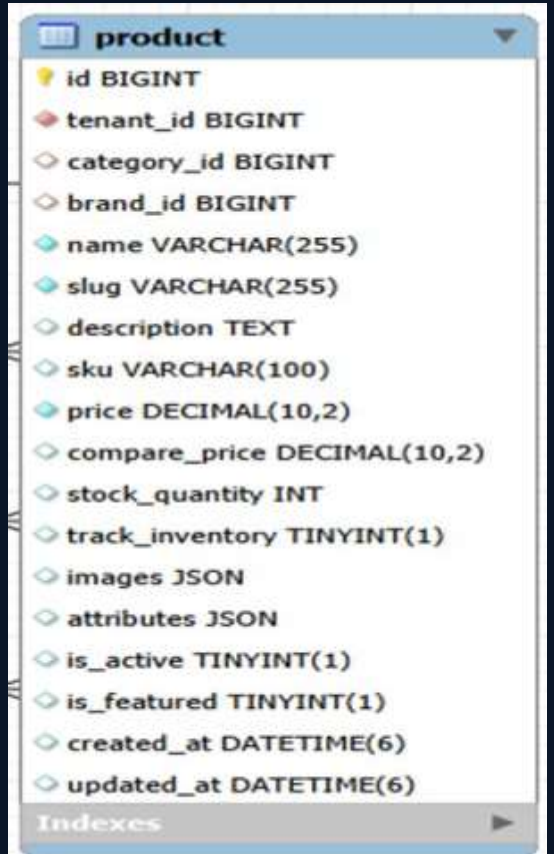
tenant_address

tenant — арендаторы (ключевая сущность)

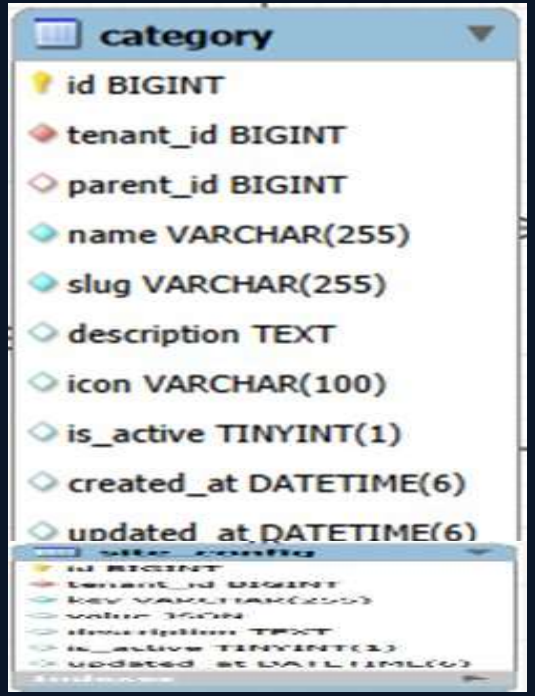
user_profile расширяет auth_user через user_id (FK). Арендаторы хранят рабочие часы и функции в полях типа JSON.

СТРУКТУРА ТАБЛИЦ: КАТАЛОГ ТОВАРОВ

product — товары



category — категории



site_config — конфигурация сайта арендатора через пары key-value с типом JSON

brand — бренды

product содержит JSON-поля images и attributes — гибкое хранение без доп. таблиц

category.parent_id → self-reference — иерархическая структура категорий

Каждая запись привязана к арендатору через tenant_id (мультитенантность)

СТРУКТУРА ТАБЛИЦ: КОРЗИНА, ЗАКАЗЫ, ПЛАТЕЖИ

Корзина

cart

- id BIGINT
- tenant_id BIGINT
- user_id BIGINT
- session_key VARCHAR(255)
- created_at DATETIME(6)
- updated_at DATETIME(6)

Indexes

cart + cart_item

cart_item

- id BIGINT
- cart_id BIGINT
- product_id BIGINT
- quantity INT
- price DECIMAL(10,2)

Indexes

Заказы

order

- id BIGINT
- tenant_id BIGINT
- user_id BIGINT
- order_number VARCHAR(50)
- customer_name VARCHAR(254)
- customer_email VARCHAR(254)
- customer_phone VARCHAR(20)
- shipping_address VARCHAR(500)
- shipping_city VARCHAR(100)
- shipping_postal_code VARCHAR(20)
- subtotal DECIMAL(10,2)
- shipping_cost DECIMAL(10,2)
- total DECIMAL(10,2)
- status VARCHAR(20)
- created_at DATETIME(6)
- updated_at DATETIME(6)
- paid_at DATETIME(6)

Indexes

order_item

order_item

- id BIGINT
- order_id BIGINT
- product_id BIGINT
- product_name VARCHAR(255)
- product_sku VARCHAR(100)
- quantity INT
- price DECIMAL(10,2)
- subtotal DECIMAL(10,2)

Indexes

💡 order хранит снимок данных покупателя (customer_name, email, phone) — защита от изменений профиля после оформления заказа

payment

payment

- id BIGINT
- tenant_id BIGINT
- order_id BIGINT
- payment_method VARCHAR(20)
- status VARCHAR(20)
- amount DECIMAL(10,2)
- transaction_id VARCHAR(255)
- created_at DATETIME(6)
- completed_at DATETIME(6)

Indexes

notification

notification

- id BIGINT
- tenant_id BIGINT
- user_id BIGINT
- type VARCHAR(20)
- title VARCHAR(255)
- message TEXT
- is_read TINYINT(1)
- created_at DATETIME(6)

Indexes

ЖИЗНЕННЫЙ ЦИКЛ ЗАКАЗА

Ожидание
Создан, не оплачен
pending

Подтверждён
Оплата прошла
confirmed

В пути
Передан логисту
shipped

На ПВЗ
Прибыл, ждёт выдачи
delivered

Выдан
Завершён успешно
completed

✕ cancelled
из любого статуса кроме completed

Хранимая процедура cancel_order():

- Проверяет допустимость отмены (нельзя отменить completed)
- В транзакции: меняет статус заказа, отменяет платёж, создаёт уведомление покупателю
- Откат при любой ошибке — гарантия целостности данных

РЕАЛИЗАЦИЯ БИЗНЕС-ЛОГИКИ СРЕДСТВАМИ MySQL

Триггеры (Triggers)

- trg_before_product_update – не даёт изменить остаток товара на отрицательный

Хранимые процедуры

- sp_cancel_order – отмена товара

Представления (Views)

- vw_pvz_parcel - удобного мониторинга и анализа посылок

Роли и безопасность

- Роль db_admin— Права администратора — полный доступ
- Роль db_manager— Права менеджера — управление товарами, заказами, уведомлениями
- db_customer - просмотр каталога, работа с корзиной и своими заказами

ТРЕБОВАНИЯ К БД И НОРМАЛИЗАЦИЯ

Функциональные требования

Хранение данных

Тенанты, пользователи, товары, заказы, платежи, корзины, уведомления

Изоляция данных

Каждая запись привязана к tenant_id — логическая изоляция арендаторов

Жизненный цикл заказа

Полная цепочка статусов со сменой состояний и откатом

Ссылочная целостность

FK, NOT NULL, UNIQUE — автоматический контроль консистентности

Мониторинг ПВЗ

Выявление посылок на ПВЗ >7 дней через представление БД

Технологический стек

MySQL 8.0 • MySQL Workbench • Django 5.x • Django ORM
Проект Django Marketplace на GitHub: github.com/Bibuk/PVZ-Constructor

Нормализация до 3NF

1НФ Все атрибуты атомарны, нет повторяющихся групп. Массивы изображений → JSON-поле images.

2НФ Все не-ключевые атрибуты зависят от всего первичного ключа, не от его части.

3НФ Нет транзитивных зависимостей. Бренды и категории вынесены в отдельные таблицы.

Нефункциональные требования

Связи «один ко многим». Транзакционная безопасность (ACID). Масштабируемость через индексы. Поддержка большого числа одновременных запросов.

РЕЗУЛЬТАТЫ ПРОЕКТА

14
таблиц
нормализованных до 3NF

8
бизнес-процессов
реализованы средствами БД

3
триггера
автоматический контроль событий

3+
процедуры
транзакционная бизнес-логика

Выполненные задачи

- ✓ Проанализированы бизнес-процессы мультитенантной платформы ПВЗ
- ✓ Сформулированы функциональные и нефункциональные требования к БД
- ✓ Обоснован выбор СУБД MySQL на основе сравнительного анализа
- ✓ Разработана ER-диаграмма (MySQL Workbench), 14 таблиц, нормализация 3NF
- ✓ Реализованы триггеры, хранимые процедуры, представления, роли
- ✓ Выполнено полное тестирование с тестовыми данными
- ✓ БД интегрирована в реальный продукт Django Marketplace (GitHub)

Практическая ценность

Разработанная БД является фундаментом реально функционирующего программного продукта — платформы «Django Marketplace». Исходный код проекта опубликован в открытом репозитории на GitHub.

Архитектура системы позволяет при необходимости перейти на PostgreSQL или интегрировать дополнительные аналитические решения без изменения бизнес-логики.

🔗 github.com/Bibuk/PVZ-Constructor

ЗАКЛЮЧЕНИЕ

Все поставленные задачи выполнены в полном объёме

Цель достигнута: разработана реляционная БД для мультитенантной платформы конструктора ПВЗ «Django Marketplace»

Глава 1. Анализ предметной области

Описаны ключевые бизнес-процессы. Сформулированы функциональные и нефункциональные требования. Обоснован выбор MySQL.

Глава 2. Проектирование и реализация

ER-диаграмма (MySQL Workbench). 14 таблиц. Нормализация 3NF. Реализованы триггеры, процедуры, представления, роли.

Тестирование

Все механизмы проверены на тестовых данных. Транзакционная безопасность подтверждена.

Практическая ценность

БД — основа реального продукта Django Marketplace, доступного на GitHub. Готова к развитию.